Plataforma de Feedback de Aulas

Tech Challenge — Fase 4 · PosTech

Integrante	EBER SPULVEDA LASSO - RM: 371982
Repositório (público)	https://github.com/tech-challenge-fase4/feedback-platform
Vídeos de demonstração	ver seção 10
Data de entrega	17/11/2026

Sumário

1. Visão geral da solução
2. Modelo de cloud escolhido
3. Arquitetura da solução
4. Documentação das funções
5. Qualidade do código
6. Configuração do ambiente de nuvem
7. Segurança e governança
8. Monitoramento
9. Instruções de deploy
10. Vídeos de demonstração
11. Evidências de funcionamento

---

1. Visão geral da solução

Estudantes enviam avaliações das aulas, cada uma com uma descrição e uma nota de 0 a 10. A plataforma:

1. **recebe e grava** cada avaliação pelo endpoint `POST /avaliacao`;
2. **avisa os administradores por e-mail na hora** quando a avaliação é crítica;
3. **envia toda segunda-feira às 08:00** (horário de Brasília) um **relatório semanal** com a média das notas, a quantidade de avaliações por dia e por urgência, e a lista das avaliações.

Regra de urgência

O enunciado pede a classificação por urgência mas não define a regra. Adotamos esta:

Nota	Urgência	Ação
0 a 3	**ALTA** (crítica)	E-mail imediato ao administrador + relatório semanal
4 a 6	**MEDIA**	Só no relatório semanal
7 a 10	**BAIXA**	Só no relatório semanal

A regra fica em um único lugar do código (`domain/Urgencia.java`).

---

2. Modelo de cloud escolhido

Aspecto	Escolha
Modelo de implantação	Nuvem **pública**: AWS, região `us-east-1` (N. Virginia)
Modelo de serviço	**Serverless**: FaaS (**AWS Lambda**) + serviços gerenciados (API Gateway, DynamoDB, SNS, EventBridge Scheduler, CloudWatch, X-Ray)
Estilo de arquitetura	**Orientada a eventos**: uma requisição HTTP, uma gravação no banco e um horário agendado disparam as funções

Por que serverless

- **A carga é irregular:** há picos logo depois das aulas e quase nenhum acesso de madrugada. O Lambda escala automaticamente de zero a muitas execuções simultâneas e volta a zero.
- **Pagamento só pelo uso:** sem requisições, o custo é praticamente zero. Isso importa num projeto acadêmico com créditos limitados.
- **Sem servidores para operar:** a equipe não gerencia sistema operacional, patches nem capacidade, e se concentra na regra de negócio.
- **Responsabilidade única por função:** cada função tem gatilho, permissões, logs e alarmes próprios, e pode ser atualizada e monitorada separadamente.

Responsabilidade compartilhada

AWS cuida de (segurança da nuvem)	A equipe cuida de (segurança na nuvem)
Hardware, rede física, hipervisor	Permissões IAM (quem acessa o quê)
Runtime Java do Lambda e seus patches	Código das funções e validação de entrada
Disponibilidade e replicação do DynamoDB	Dados gravados e o que vai para os logs
Infraestrutura do API Gateway e do SNS	Configuração de chave de API, limites e gatilhos

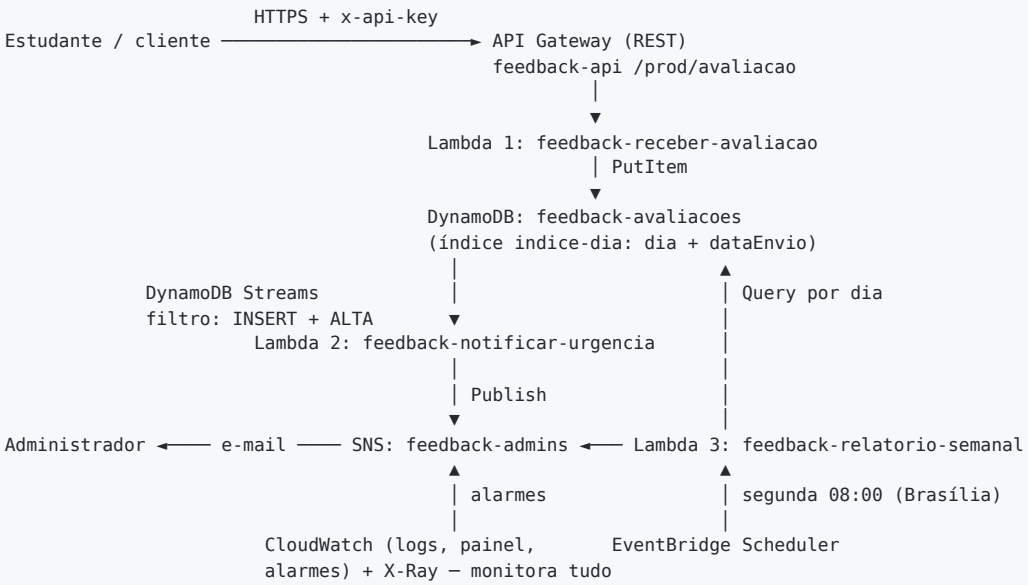
Componentes envolvidos

Componente	Serviço AWS	Papel na solução
Entrada HTTP	**API Gateway** (REST) `feedback-api`, stage `prod`	Expõe `POST /avaliacao` por HTTPS, com chave de API e plano de uso
Função 1	**Lambda** `feedback-receber-avaliacao`	Valida a avaliação, calcula a urgência e grava
Banco de dados	**DynamoDB** `feedback-avaliacoes` + índice `indice-dia`	Armazena as avaliações; modo sob demanda; criptografado em repouso
Evento de gravação	**DynamoDB Streams**	Publica cada item novo; um filtro só deixa passar urgência ALTA
Função 2	**Lambda** `feedback-notificar-urgencia`	Envia o alerta de avaliação crítica
Agendamento	**EventBridge Scheduler** `feedback-relatorio-semanal-agenda`	Dispara o relatório toda segunda às 08:00 (America/Sao_Paulo)
Função 3	**Lambda** `feedback-relatorio-semanal`	Consolida a semana e envia o relatório
Notificações	**SNS** `feedback-admins`	Entrega os e-mails aos administradores
Observabilidade	**CloudWatch** (logs, painel, alarmes) + **X-Ray**	Monitora as funções e a API e alerta em caso de erro
Governança	**IAM**, **CloudTrail**, **AWS Budgets**	Controle de acesso, auditoria e controle de custos

---

3. Arquitetura da solução

3.1 Arquitetura de nuvem



3.2 Fluxos

Fluxo 1: registrar avaliação (síncrono)

1. O cliente chama `POST /avaliacao` com o header `x-api-key`.
2. O API Gateway valida a chave e o limite de uso e repassa a requisição à Função 1 (integração *Lambda proxy*).
3. A Função 1 valida os dados, calcula a urgência, grava no DynamoDB e responde **201** com `id`, `urgencia` e `dataEnvio`.

Fluxo 2: alerta de avaliação crítica (assíncrono)

1. A gravação gera um evento no **DynamoDB Streams**.
2. O gatilho da Função 2 tem um **filtro** (`eventName = INSERT` e `urgencia = ALTA`). Avaliações não críticas nem chegam a executar a função, o que economiza execuções.
3. A Função 2 publica no **SNS**, que envia o e-mail **[URGENTE]** com descrição, urgência e data de envio.

Fluxo 3: relatório semanal (agendado)

1. O **EventBridge Scheduler** executa a Função 3 toda segunda às 08:00 (`cron(0 8 ? * MON *)`, fuso `America/Sao_Paulo`).
2. A Função 3 consulta os **7 dias anteriores** (segunda a domingo) no índice `indice-dia`, dia a dia, sem varrer a tabela inteira.
3. Ela calcula total, média, quantidade por dia e por urgência e envia o relatório pelo **SNS**.

3.3 Modelo de dados (DynamoDB)

Tabela `feedback-avaliacoes`, chave de partição `id` (String):

Atributo	Tipo	Exemplo	Observação
`id`	String	`3f2c…` (UUID)	Chave primária
`descricao`	String	`Aula excelente, muito didática`	1 a 1000 caracteres
`nota`	Number	`9`	Inteiro de 0 a 10
`urgencia`	String	`BAIXA`	Calculada pela nota
`dataEnvio`	String	`2026-11-02T13:10:00Z`	Instante UTC (ISO-8601)
`dia`	String	`2026-11-02`	Data local de São Paulo; partição do índice

Índice global `indice-dia`: partição `dia` + ordenação `dataEnvio`. Permite buscar as avaliações de um dia com `Query`, que é mais barata e rápida que `Scan`.

3.4 Arquitetura de software: Clean Architecture + SOLID

O código é um único projeto Java 25 / Quarkus, organizado em camadas:

```
src/main/java/br/com/techchallenge/feedback/
├── domain/              Regras de negócio puras (só Java)
│                        Avaliacao, Urgencia, PeriodoRelatorio, RelatorioSemanal
├── application/
│   ├── port/            Interfaces (portas): AvaliacaoRepository, NotificacaoGateway, GeradorDeId
│   ├── usecase/         Um caso de uso por operação:
│   │                    RegistrarAvaliacao, NotificarAvaliacaoCritica, GerarRelatorioSemanal
│   └── formatacao/      Texto dos e-mails: MensagemUrgenciaFormatter, RelatorioFormatter
├── infrastructure/      Implementações técnicas das portas
│                        DynamoDbAvaliacaoRepository, SnsNotificacaoGateway, UuidGeradorDeId
│                        e ConfiguracaoCasosDeUso (montagem / composition root)
└── entrypoint/lambda/   Os 3 handlers Lambda (só traduzem evento ↔ caso de uso)
```

Regra de dependência: `entrypoint` e `infrastructure` → `application` → `domain`. O domínio e os casos de uso **não importam** AWS SDK, Quarkus nem Jackson. Assim, as regras de negócio podem ser testadas e mantidas sem depender da nuvem.

Princípio SOLID	Onde aparece
S – Responsabilidade única	Cada função Lambda tem um único papel. Cada caso de uso faz uma operação. Os handlers só traduzem eventos. O cálculo (`RelatorioSemanal`) é separado da formatação (`RelatorioFormatter`). O formato do DynamoDB fica só em `AvaliacaoItemMapper`
O – Aberto/fechado	Um novo canal de aviso (ex.: Slack) seria uma nova implementação de `NotificacaoGateway`, sem alterar os casos de uso
L – Substituição de Liskov	Qualquer implementação das portas (DynamoDB, SNS ou em memória) pode ser usada pelos casos de uso
I – Segregação de interfaces	Portas pequenas, com 1 ou 2 métodos cada
D – Inversão de dependência	Os casos de uso dependem de interfaces; `ConfiguracaoCasosDeUso` entrega as implementações concretas

---

4. Documentação das funções

As 3 funções usam o **mesmo pacote** (`target/function.zip`, gerado pelo Quarkus) e o **mesmo handler de entrada**:

```
io.quarkus.amazon.lambda.runtime.QuarkusStreamHandler::handleRequest
```

A variável de ambiente `QUARKUS_LAMBDA_HANDLER` escolhe qual classe `@Named` cada função executa. Na AWS são **3 recursos independentes**, cada um com gatilho, role IAM, logs e alarme próprios.

Configuração comum: runtime **Java 25**, arquitetura **arm64**, **1024 MB** de memória, timeout de **30 s**, rastreamento **X-Ray** ativo, armazenamento efêmero de 512 MB (padrão).

4.1 `feedback-receber-avaliacao`: registrar avaliação

Responsabilidade	Validar a entrada, calcular a urgência e gravar a avaliação
Gatilho	API Gateway: `POST /avaliacao` (header `x-api-key` obrigatório)
`QUARKUS_LAMBDA_HANDLER`	`receberAvaliacao`
Classes	`ReceberAvaliacaoHandler` → `RegistrarAvaliacaoUseCase` → `DynamoDbAvaliacaoRepository`
Variáveis de ambiente	`TABELA_AVALIACOES=feedback-avaliacoes`
Permissão IAM	Somente `dynamodb:PutItem` na tabela `feedback-avaliacoes`

Requisição

```
POST /prod/avaliacao
Content-Type: application/json
x-api-key: <chave>

{ "descricao": "Aula excelente, muito didática", "nota": 9 }
```

Respostas

Status	Quando	Corpo
201	Avaliação gravada	`{"id": "...", "urgencia": "BAIXA", "dataEnvio": "2026-11-02T13:10:00Z", "mensagem": "Avaliação recebida com sucesso"}`
400	Corpo ausente ou JSON inválido; `descricao` ausente, vazia ou com mais de 1000 caracteres; `nota` ausente, fora de 0–10, decimal ou em texto (`"9"`)	`{"erro": "<motivo>"}`
403	Sem `x-api-key` ou chave inválida (barrado pelo API Gateway)	`{"message": "Forbidden"}`
429	Limite de taxa ou cota do plano de uso excedido (barrado pelo API Gateway)	`{"message": "..."}`
500	Erro inesperado	`{"erro": "Erro interno ao processar a avaliação"}`

4.2 `feedback-notificar-urgencia`: alerta de avaliação crítica

Responsabilidade	Avisar os administradores por e-mail sobre avaliações críticas
Gatilho	DynamoDB Streams da tabela (*New image*), lote de 10, posição *Latest*, 3 tentativas, filtro `{"eventName":["INSERT"],"dynamodb":{"NewImage":{"urgencia":{"S":["ALTA"]}}}}`
`QUARKUS_LAMBDA_HANDLER`	`notificarUrgencia`
Classes	`NotificarUrgenciaHandler` → `NotificarAvaliacaoCriticaUseCase` → `SnsNotificacaoGateway`
Variáveis de ambiente	`TOPICO_ARN=<ARN do tópico feedback-admins>`
Permissões IAM	`sns:Publish` no tópico; `DescribeStream`, `GetRecords` e `GetShardIterator` no stream da tabela; `ListStreams`
Saída	E-mail **“[URGENTE] Nova avaliacao critica recebida”** com descrição, urgência e data de envio

A regra “só avisa se a urgência for ALTA” está **no caso de uso**, além do filtro do gatilho. Assim ela é testável e não depende só da configuração da AWS.

4.3 `feedback-relatorio-semanal`: relatório semanal

Responsabilidade	Consolidar as avaliações da semana e enviar o relatório
Gatilho	EventBridge Scheduler `feedback-relatorio-semanal-agenda`: `cron(0 8 ? * MON *)`, fuso `America/Sao_Paulo`, payload `{}`
`QUARKUS_LAMBDA_HANDLER`	`relatorioSemanal`
Classes	`RelatorioSemanalHandler` → `GerarRelatorioSemanalUseCase` → `DynamoDbAvaliacaoRepository` + `SnsNotificacaoGateway`
Variáveis de ambiente	`TABELA_AVALIACOES=feedback-avaliacoes`, `TOPICO_ARN=<ARN do tópico>`
Permissões IAM	`dynamodb:Query` **somente** no índice `indice-dia`; `sns:Publish` no tópico
Período	Os 7 dias anteriores à execução (na segunda: de segunda a domingo da semana anterior)
Saída	E-mail **“Relatorio semanal de feedbacks - dd/MM/aaaa a dd/MM/aaaa”** com total, média das notas, quantidade por dia, quantidade por urgência e a lista das avaliações (descrição, urgência e data de envio)

Para execução manual fora do agendamento, a função aceita `{"incluirHoje": true}`, que cobre os 6 dias anteriores mais o dia atual.

4.4 Variáveis de ambiente (resumo)

Variável	Função 1	Função 2	Função 3	Valor
`QUARKUS_LAMBDA_HANDLER`	✔	✔	✔	`receberAvaliacao` / `notificarUrgencia` / `relatorioSemanal`
`TABELA_AVALIACOES`	✔		✔	`feedback-avaliacoes`
`TOPICO_ARN`		✔	✔	ARN do tópico `feedback-admins`
`JAVA_TOOL_OPTIONS`	✔	✔	✔	`-XX:+TieredCompilation -XX:TieredStopAtLevel=1` (acelera a partida a frio)

---

5. Qualidade do código

- **Clean Architecture + SOLID** (seção 3.4): regras de negócio isoladas da infraestrutura.
- **Entidade sempre válida:** o construtor de `Avaliacao` valida tudo (nota de 0 a 10, descrição de 1 a 1000 caracteres, urgência coerente com a nota). Não existe avaliação inválida em memória.
- **Validação estrita da entrada:** `"9"` (texto) e `9.5` (decimal) são recusados como nota, em vez de convertidos silenciosamente.
- **Erros tratados com status HTTP corretos** (400, 500) e mensagens claras em português.
- **Privacidade nos logs:** o texto da avaliação **não** vai para os logs; registram-se apenas `id`, nota e urgência.
- **Documentação no código:** todas as classes têm Javadoc explicando a responsabilidade e o papel na arquitetura.
- **Nomes do domínio em português**, alinhados ao enunciado (`Avaliacao`, `Urgencia`, `RelatorioSemanal`).
- **Testes automatizados:** domínio, casos de uso (com repositório e notificação em memória), formatação dos e-mails, handlers e uma verificação automática da regra de dependência entre camadas (`ArquiteturaTest`). Executar com `./mvnw test`.

---

6. Configuração do ambiente de nuvem

Todos os recursos estão em `us-east-1` e foram criados pelo Console AWS, nesta ordem.

#	Recurso	Configuração
1	**DynamoDB** `feedback-avaliacoes`	Partição `id` (String); capacidade **On-demand**; criptografia em repouso (*owned by Amazon DynamoDB*); índice global `indice-dia` (`dia` + `dataEnvio` , projeção *All*); **Streams** ligado com *New image*
2	**SNS** `feedback-admins`	Tópico *Standard*; assinatura de e-mail do administrador confirmada
3	**IAM** (3 roles)	`feedback-role-receber` , `feedback-role-notificar` , `feedback-role-relatorio` : cada uma com `AWSLambdaBasicExecutionRole` + `AWSXRayDaemonWriteAccess` + 1 política *inline* de menor privilégio (seção 7.2)
4	**Lambda** (3 funções)	Java 25, arm64, 1024 MB, 30 s, X-Ray ativo, handler `QuarkusStreamHandler` , variáveis de ambiente da seção 4.4
5	**Gatilho do stream**	DynamoDB → `feedback-notificar-urgencia` : lote 10, *Latest*, 3 tentativas, filtro `INSERT` + `ALTA`
6	**API Gateway** `feedback-api`	REST regional; recurso `/avaliacao` ; método `POST` com *Lambda proxy integration*; **API key required**; stage `prod`
7	**Plano de uso** `feedback-plano-de-uso`	10 req/s, rajada de 20, cota de 1000 req/dia; chave `feedback-chave-demo` associada ao stage `prod`
8	**EventBridge Scheduler** `feedback-relatorio-semanal-agenda`	Recorrente, `cron(0 8 ? * MON *)` , `America/Sao_Paulo` , *flexible time window* desligado, alvo `feedback-relatorio-semanal` , payload `{}`
9	**CloudWatch**	Retenção de logs de 30 dias, painel `feedback-painel` e 4 alarmes (seção 8)

Por que REST API e não HTTP API no API Gateway: só a REST API oferece **chave de API e plano de uso** (limite de taxa e cota), que fazem parte da proteção do endpoint público.

Por que arm64: no Lambda, o processador ARM (Graviton) custa menos por milissegundo e tem bom desempenho para Java, sem nenhuma mudança no código.

---

7. Segurança e governança

7.1 Acesso à conta

- **MFA** no usuário root e nos usuários IAM.
- O **root não é usado no dia a dia**. Usamos o usuário IAM `admin-techchallenge` e o login pela URL do alias da conta.
- **Grupos IAM:** `Administradores` (`AdministratorAccess`) para quem altera recursos e `Observadores` (`ReadOnlyAccess`) para quem só consulta.
- **CloudTrail** registra todas as ações na conta (histórico de 90 dias), garantindo rastreabilidade.
- **AWS Budgets** com orçamento “Zero spend”: alerta por e-mail ao primeiro gasto.

7.2 Menor privilégio por função

Cada função tem **uma role própria**, com somente as ações e os recursos de que precisa. Tudo o que não está explicitamente permitido é negado.

Role	Permite	Não permite
`feedback-role-receber`	`dynamodb:PutItem` na tabela	Ler, apagar, consultar, publicar no SNS
`feedback-role-notificar`	`sns:Publish` no tópico; leitura do stream da tabela	Gravar, alterar ou consultar a tabela
`feedback-role-relatorio`	`dynamodb:Query` **só no índice** `indice-dia` ; `sns:Publish` no tópico	Gravar, apagar, `Scan` , `GetItem` , ler o stream

Exemplo, a política da Função 1:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GravarAvaliacao",
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:us-east-1:<ACCOUNT_ID>:table/feedback-avaliacoes"
    }
  ]
}
```

Se a Função 1 fosse comprometida, o máximo possível seria inserir itens. Não seria possível ler as avaliações nem apagar dados.

7.3 Proteção da API

- **HTTPS** obrigatório (padrão do API Gateway).
- **Chave de API** (`x-api-key`): sem ela, a requisição é barrada com **403** antes de chegar à função.
- **Plano de uso:** 10 req/s, rajada de 20 e cota de 1000 req/dia. Limita abuso e custo mesmo se a chave vazar.
- **Validação estrita** do corpo na função (seção 4.1).

7.4 Proteção dos dados

- DynamoDB **criptografado em repouso** (padrão, sem custo).
- Avaliações **anônimas**: nenhum dado pessoal do estudante é coletado.
- O texto das avaliações **não é registrado nos logs**.
- A chave de API e o número da conta **não estão no repositório**.

---

8. Monitoramento

O quê	Configuração
Logs	CloudWatch Logs, um grupo por função (`/aws/lambda/feedback-*`), retenção de **30 dias**
Consulta de erros	CloudWatch Logs Insights sobre os 3 grupos: `filter @message like /ERROR|Erro|Exception/`
Painel	`feedback-painel`: invocações, erros e duração média das 3 funções; requisições, 4XX e 5XX da API
Alarmes	`feedback-erros-receber`, `feedback-erros-notificar`, `feedback-erros-relatorio` (métrica `Errors` ≥ 1 em 5 min) e `feedback-api-5xx` (`5XXError` ≥ 1 em 5 min). Todos notificam o administrador por e-mail via SNS `feedback-admins`
Rastreamento	**AWS X-Ray** ativo nas funções e na API: mostra o caminho API Gateway → Lambda → DynamoDB e a latência de cada etapa
Auditoria	CloudTrail (quem alterou o quê na conta)
Custos	AWS Budgets com alerta por e-mail

Os administradores recebem assim **dois tipos de notificação automática**:

- **de negócio:** avaliação crítica (Função 2);
- **técnica:** erro em uma função ou na API (alarmes do CloudWatch).

9.6 Testar a API após o deploy

```
curl -i -X POST "https://<API_ID>.execute-api.us-east-1.amazonaws.com/prod/avaliacao" \
  -H "Content-Type: application/json" \
  -H "x-api-key: <CHAVE>" \
  -d '{"descricao":"Aula excelente, muito didática","nota":9}'
```

A pasta `postman/` do repositório traz uma **collection do Postman** com avaliações variadas (todas as faixas de nota) e os cenários de erro, todos com testes automáticos de status e urgência.

---

10. Vídeos de demonstração

#	Conteúdo	Link
1	**Apresentação geral:** modelo de cloud, arquitetura, configurações no console AWS (funções, roles IAM, API Gateway, DynamoDB, SNS, Scheduler), segurança e monitoramento	[INSERIR LINK DO YOUTUBE]
2	**Aplicação em funcionamento:** envio de avaliações pela API, gravação no DynamoDB, e-mail de avaliação crítica, cenários de erro (400/403), logs e X-Ray	[INSERIR LINK DO YOUTUBE]
3	**Relatório semanal automático:** execução pelo EventBridge Scheduler na segunda-feira às 08:00 e o e-mail recebido	[INSERIR LINK DO YOUTUBE]

---

11. Evidências de funcionamento

Entre 30/10 e 15/11/2026 a aplicação recebeu avaliações reais pelo endpoint público, uma rodada por dia, com notas de todas as faixas. Os relatórios foram gerados pelo próprio agendamento, sem disparo manual.